

Experiment - 2

* AIM:

Convert the ER diagram into a relational schema. Write complete MySQL CREATE TABLE statements with primary keys, foreign keys, NOT NULL, UNIQUE, ON DELETE CASCADE and SET NULL constraints. Insert sample data and demonstrate referential integrity violations.

* Theory:

- Relational schema → A relational schema is the logical, table-based representation of a data. Base, obtained by systematically converting an ER diagram into a set of relations (tables), each with defined columns, data types and constraints. This conversion is called ER to - Relational mapping.

- Rules for Mapping ER diagram to Relational Schema -

▷ Strong entity :-

Becomes a table; its key attribute becomes the primary key (eg., customer, seller, category, product, order, payment, delivery).

▷ Weak entity :-

Becomes a table whose primary key is a composite key - owner entity's key + the weak entity's own partial key. A Foreign key to the owner is added, typically with ON DELETE CASCADE, since the weak entity

cannot exist without its owner. (e.g., Address depends on customer; order item depends on order).

▷ 1:1 relationship:-

The foreign key is placed on their entities either side and made **UNIQUE**, so each row on that side maps to exactly one row on the other.
(e.g. - order - payment, order delivery).

▷ 1:N relationship:-

The foreign key is placed on the 'many side', referencing the primary key of the 'one side'.
(e.g. - order item resolves the M:N relationship between order and product).

▷ M:N relationship:-

A new junction/bridge table is created. Containing foreign keys to both participating entities.
(e.g. - sellers sell many products, category classifies many products).

▷ Multi-valued attribute:-

Becomes a separate table with a foreign key back to the owning entity, since a single column cannot hold multiple values.
(e.g. - customer's phone numbers).

▷ Composite attributes:-

Its sub-parts become individual columns in the table (flattened); the composite itself is the table.
(e.g. - Name - first name, last name).

▷ Specialization/SA hierarchy :-

Each subclass becomes its own table sharing the primary key of the superclass table, linked by a foreign key (usually ON DELETE CASCADE).
A discrimination column (eg:- product type) may be kept in the superclass table to indicate ~~what~~ which subclass a row belongs to.
(eg:- product - electronics, fashions, grocery, books).

▷ Recursive/self-referencing relationship :-

The foreign key references the primary key of the same table. (eg:- category's parent category ID references category ID).

• Integrity Constraints used in the schema -

▷ Primary key constraints :-

Uniquely identifies each row, cannot be NULL.

▷ Foreign key (referential integrity) constraints :-

Ensures a value in one table matches an entity existing value in another preventing orphan/dangling references.

▷ NOT NULL constraints :-

Forces a column to always have a value (used for mandatory attributes reflecting total participation).

▷ UNIQUE constraints :-

Ensures no duplicate values in a column (eg:- Email list number), and is used to enforce 1:1 cardinality on a foreign key.

• Referential actions on DELETE/UPDATE —

▷ CASCADE :-

Automatically deletes/updates dependent rows where the parent row is deleted/updated used for weak entities and specialization subtypes which have no independent existence.

▷ SET NULL :-

Sets the foreign key to NULL when the parent is deleted, letting the dependent row survive used where the relationship is optional
(eg:- product can exist without a category.)

The ER model captures business rules conceptually (entities, relationships, constraints) but a DBMS can only store and query data in tables. This mapping process translates that conceptual design into an implemented relational schema which preventing all the original semantics — keys stay unique, weak entities stay dependent, optional relationships stay optional — which is then enforced automatically by the database engine through the constraints by the database declared in the (CREATE TABLE).

* Source code :

```
DROP DATABASE IF EXISTS Company-db;  
CREATE DATABASE Company-db;  
USE Company-db;
```

- Department Table:- [Relationship: Department (1): (N) Employee]

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY AUTO-INCREMENT,  
    dept_name VARCHAR(50) NOT NULL UNIQUE  
);
```

- Employee Table:- [Employee has a self-referring manager relationship]

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY AUTO-INCREMENT,  
    emp_name VARCHAR(50) NOT NULL,  
    email VARCHAR(50) NOT NULL UNIQUE,  
    dept_id INT,  
    dept_name INT,  
    FOREIGN KEY (dept_id)  
        REFERENCES Department (dept_id)  
        ON DELETE CASCADE,  
    FOREIGN KEY (emp_id)  
        REFERENCES Employee (emp_id)  
        ON DELETE SET NULL  
);
```

- Project Table:- [Each project optionally belongs to a department]

CREATE TABLE Project (

proj-id INT PRIMARY KEY AUTO-INCREMENT,

proj-name VARCHAR (50) NOT NULL UNIQUE,

dept-id INT,

FOREIGN KEY (dept-id)

REFERENCES Department (dept-id)

ON DELETE SET NULL

);

- Location Table:- [Each department has exactly one location]

CREATE TABLE Location (

loc-id INT PRIMARY KEY AUTO-INCREMENT,

city VARCHAR (50) NOT NULL,

dept-id INT ~~NOT~~ NOT NULL UNIQUE,

FOREIGN KEY (dept-id)

REFERENCES Department (dept-id),

ON DELETE CASCADE

);

- Emp-Project Table:- [M:N relationship between Employee and Project]

CREATE TABLE emp-project (

emp-id INT NOT NULL,

proj-id INT NOT NULL,

role VARCHAR (30) NOT NULL,

PRIMARY KEY (emp-id, proj-id),

FOREIGN KEY (emp-id)

REFERENCES Employee (emp-id),

ON DELETE CASCADE,

FOREIGN KEY (proj-id)

REFERENCES Project (proj_id)
ON DELETE CASCADE
);

• Sample Data:-

INSERT INTO Department VALUES
(1, 'IT'), (2, 'HR');

INSERT INTO Employee VALUES
(001, 'Amit', 'amit@co.com', 1, NULL),
(002, 'Neha', 'neha@co.com', 1, 1),
(003, 'Ravi', 'ravi@co.com', 2, NULL),
(004, 'Priya', 'priya@co.com', 2, 3);

INSERT INTO Project VALUES
(01, 'ERP upgrade', 1),
(02, 'Payroll rename', 2);

INSERT INTO Location VALUES
(101, 'Lucknow', 1), (102, 'Delhi', 2);

INSERT INTO emp-project VALUES
(001, 01, 'Lead'),
(002, 01, 'Developer'),
(003, 02, 'Lead'),
(004, 02, 'Developer');

• Display Tables:-

SELECT * FROM Department;
SELECT * FROM Employee;
SELECT * FROM Project;
SELECT * FROM Location;
SELECT * FROM emp-project;

* OUTPUT:-

Department	
dept-id	dept-name
1	IT
2	HR

Employee				
emp-id	emp-name	email	dept-id	dept-id
001	Amit	amit@co.com	1	NULL
002	Neha	neha@co.com	1	1
003	Ravi	ravi@co.com	2	NULL
004	Priya	priya@co.com	2	3

Project		
proj-id	proj-name	dept-id
01	ERP upgrade	1
02	Payroll remain	2

Location		
loc-id	city	dept-id
101	Lucknow	1
102	Delhi	2

emp-project		
emp-id	proj-id	role
001	01	Lead
002	01	Developer
003	02	Lead
004	02	Developer